Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «Криптография и сетевая безопасность»
Тема: **«Избыточное кодирование данных в информационных системах. Код Хемминга. Итеративные коды»**

Выполнил:
Студент 4 курса
Группы АС-65
Ярмак К. А.
Проверил:
Хацкевич А. С.

Брест 2026

Цель: приобретение практических навыков кодирования/де- кодирования двоичных данных при использовании кода Хемминга. приобретение практических навыков кодирования/дс- кодирования двоичных данных при использовании итеративных кодов .

Вариант 12

Часть 1

1. На основе информационного сообщения, представленного символами рус-ского/английского алфавитов, служебными символами и цифрами, содержащегося в некоторт текстовом файле, сформировать информационнос сообщение в двоич-ном виде; длина сообщения в бинарном виде должна быть не менее 16 символов. Для выполнения этого задания можно использовать коды ASCII символов алфавита либо результаты лабораторной работы N3. 2. Для полученного информационного слова проверочную матрицу Хемминга. 3. Используя построенную матрицу, вычислить избыточные символы (слово X_r). 4. Принять исходное слово следующим числом ошибок: 0, 1, 2. Позиция ошибки определяется (генерируется) случайным образом. 5. Для полученного слова $Y_n = Y_k$, Y_r, используя уже известную про-верочную матрицу Хемминга, вновь вычислить избыточные символы (обозначим их Y_r), используя выражение (4.6). 6. Вычислить и проанализировать синдром. В случае, если анализ синдрома показал, что информационное сообщение было передано с ошибкой (или 2 ошибками), сгенирировать унарный вектор ошибки $E_n = e_1, e_2, ..., e_n$ и исправить одиночную ошибку, используя формулу (4.7); проана-лизировать ситуацию при возник- новении ошибки в 2 битах. 7. Результаты оформить в виде шчега по установленным правилам. Файл анализируемой информации (а соответственно и интерфейс приложения) должен содержать исходное информационное сообщение, значения величин k, r, n, проверочную матрицу Хемминга $H_{n,k}$, слово X_n, X_r, Y_n,Y_r, Y_r', синдром S, вектор ошибки E_n. Программа не должна быть чувствительна к длине информационного сообщения.

Часть 2

Разработать собственное приложение, которое позволяет выполнять следующие операции: 1) вписывать произвольное двоичное представление информационного слова X_k (кодируемой информации) длиной k битов в двумерную матрицу раз-мерностью в соответствии с вариантом либо в трехмерную матрицу в соответствии с вариантом (указаны в табл. 5.2); 2) вычислять проверочные биты (биты па-ритетов): а) по двум; б) по трем; в) по четырем направлениям (группам паритетов); 3) формировать кодовое слово X_n, присоединением избыточных символов к информационному слову; 4) генерировать ошибку произвольной кратности (i, $i > 0$), распределенную случайным (фазом среди символов слова X_n, в результате чего формируется кодовое слово Y_n; 5) определять местоположение ошибочных символов итеративным кодом в слове Y_n в соответствии с исполызусмыми груп-

пами паритетов по пункту (2) и исправлять ошибочные символы (результат исправления слово Y_n'); 6) выполнять анализ корректирующей способности используемого кода (количественная оценка) путем сравнения соответствующих слов X_n, и Y_n' ; результат анализа может быть представлен в виде отношения общего числа сгенерированных кодовых слов с ошибками определенной одинаковой кратности (с одной ошибкой, с двумя ошибками и т. д.) к числу кодовых слов, содержащих ошибки этой кратности, которые правильно обнаружены и которые правильно скорректированы.

Вариант 12

Варинат	Длинна информационного слова (бит), k	k_1	k_2	z	Количество групп паритетов
2	20	4	5	-	2; 3
		2	10	-	2; 3
		2	5	2	2; 3; 4; 5
		2	2	5	2; 3; 4; 5

main.py:

```
import random
import sys
from itertools import product

s = lambda v: ''.join(map(str, v))


# ============================================================ p1
def text_to_bits(text):
    return [int(b) for byte in text.encode('utf-8') for b in format(byte, '08b')]

def hamming_h(k):
    r = 1
    while 2 ** r < k + r + 1:
        r += 1

    n = k + r

    H = [[(j >> i) & 1 for j in range(1, n + 1)] for i in range(r)]

    return r, n, H

def parity(H, word):
    return [sum(h & w for h, w in zip(row, word)) % 2 for row in H]

def part1(path):
    bits = text_to_bits(open(path, encoding='utf-8').read().strip('\n'))

    k = len(bits)
    if k < 16:
        sys.exit('Сообщение в бинарном виде должно быть не менее 16 бит')

    r, n, H = hamming_h(k)
```

```
    par_pos = [2 ** i - 1 for i in range(r)]

    xn = [0] * n
    it = iter(bits)
    for j in range(n):
        if j not in par_pos:
            xn[j] = next(it)

    xr = parity(H, xn)
    for i, p in enumerate(par_pos):
        xn[p] = xr[i]

    print(f'Сообщение Xk: {s(bits)}')
    print(f'k={k} r={r} n={n}')
    print('Матрица H:')
    for row in H:
        print(' ', s(row))
    print(f'Xn: {s(xn)}\nXr: {s(xr)}')

    for t in (0, 1, 2):
        en = [0] * n
        for p in random.sample(range(n), t):
            en[p] = 1
        yn = [a ^ b for a, b in zip(xn, en)]
        yr = [yn[p] for p in par_pos]
        tmp = yn[:]
        for p in par_pos:
            tmp[p] = 0
        yr2 = parity(H, tmp)
        syn = [a ^ b for a, b in zip(yr, yr2)]
        pos = sum(b << i for i, b in enumerate(syn))
        fixed = yn[:]
        if pos:
            if pos <= n:
                fixed[pos - 1] ^= 1
            ev = [0] * n
            if pos <= n:
                ev[pos - 1] = 1
        else:
            ev = [0] * n
        print(f'\n--------- ошибок: {t}')
        print(f'Yn: {s(yn)}\nYr: {s(yr)}\nYr\': {s(yr2)}')
        print(f'Синдром S: {s(syn)} (pos {pos})')
        print(f'Вектор ошибки En: {s(ev)}')
        print(f'Исправлено: {s(fixed)}; совпало с Xn: {fixed == xn}')


# ============================================================ p2
DIRS2 = [(0, 1), (1, 0), (1, 1), (1, -1)]
DIRS3 = [(0, 1, 0), (1, 0, 0), (0, 0, 1), (1, 1, 1), (1, -1, 1)]


def build_lines(dims, dirs):
    cells = list(product(*[range(d) for d in dims]))
    idx = {c: i for i, c in enumerate(cells)}
    lines = []
    for d in dirs:
        for c in cells:
            prev = tuple(a - b for a, b in zip(c, d))
            if prev in idx:
                continue
            line, cur = [], c
            while cur in idx:
```

```
                line.append(idx[cur])
                cur = tuple(a + b for a, b in zip(cur, d))
            lines.append(line)
    return lines


def encode(info, lines):
    return info + [sum(info[i] for i in l) % 2 for l in lines]


def decode(y, k, lines, cl):
    y = y[:]
    det = False
    for _ in range(len(y)):
        bad = {j for j, l in enumerate(lines) if (y[k + j] + sum(y[i] for i in
l)) % 2}
        if not bad:
            break
        det = True
        if len(bad) == 1:
            y[k + next(iter(bad))] ^= 1
            continue
        cand = [i for i in range(k) if cl[i] <= bad]
        if len(cand) != 1:
            break
        y[cand[0]] ^= 1

    return y, det


def part2(k=20, trials=10000, max_err=6):
    variants = [((4, 5), (2, 3)), ((2, 10), (2, 3)), ((2, 5, 2), (2, 3, 4, 5)),
((2, 2, 5), (2, 3, 4, 5))]
    for dims, groups in variants:
        for g in groups:
            dirs = (DIRS2 if len(dims) == 2 else DIRS3)[:g]
            lines = build_lines(dims, dirs)
            cl = [{j for j, l in enumerate(lines) if i in l} for i in range(k)]
            xk = [random.randint(0, 1) for _ in range(k)]
            xn = encode(xk, lines)
            n = len(xn)
            print(f'\nМатрица {"x".join(map(str, dims))}, групп паритетов: {g},
n={n}')

            for i in range(1, min(max_err, n) + 1):
                det = cor = 0
                for _ in range(trials):
                    y = xn[:]
                    for p in random.sample(range(n), i):
                        y[p] ^= 1

                    fixed, d = decode(y, k, lines, cl)
                    det += d
                    cor += fixed == xn
                print(f'  ошибок {i}: всего {trials}, обнаружено {det} ({det /
trials:.3f}), исправлено {cor} ({cor / trials:.3f})')


if __name__ == '__main__':
    print('============================================================ p1')
    part1(sys.argv[1] if len(sys.argv) > 1 else 'input.txt')
```

```
    print('\n============================================================ p2')
    part2()
```

```
Сообщение Xk: 01101000011001010110110001101100011011111
k=40 r=6 n=46
Матрица H:
  10101010101010101010101010101010101010101010101010
  01100110011001100110011001100110011001100110011001
  00011110000111100001111000011110000111100001111000
  00000001111111100000000111111110000000011111111
  00000000000000011111111111111110000000000000000
  00000000000000000000000000000001111111111111111
Xn: 11011101100001110010101101100011101100011011111
Xr: 111111

--------- ошибок: 0
Yn: 11011101100001110010101101100011101100011011111
Yr: 111111
Yr': 111111
Синдром S: 000000 (pos 0)
Вектор ошибки En: 00000000000000000000000000000000000000000000000000
Исправлено: 11011101100001110010101101100011101100011011111; совпало с Xn: True

--------- ошибок: 1
Yn: 11011101100001110110101101100011101100011011111
Yr: 111111
Yr': 101101
Синдром S: 010010 (pos 18)
Вектор ошибки En: 00000000000000000010000000000000000000000000000000
Исправлено: 11011101100001110010101101100011101100011011111; совпало с Xn: True

--------- ошибок: 2
Yn: 11111101000001110010101101100011101100011011111
Yr: 111111
Yr': 101011
Синдром S: 010100 (pos 10)
Вектор ошибки En: 00000000010000000000000000000000000000000000000000
Исправлено: 11111101010001110010101101100011101100011011111; совпало с Xn: False
```

Для двух ошибок p_1 и p_2 синдром равен p_1 XOR p_2. Это ненулевое число, неотличимое от синдрома одиночной ошибки в другой позиции. Поэтому обычный код Хэмминга (минимальное расстояние 3) исправляет все одиночные ошибки, а двойную ошибку не может отличить от одиночной и исправляет неверно.
Если p_1 XOR p_2 > n, указанной позиции вообще нет, и код в этом случае ничего не меняет (if pos <= n). Отличать двойные ошибки от одиночных умеет расширенный код Хэмминга с одним дополнительным общим битом чётности.

```
Матрица 4x5, групп паритетов: 2, n=29
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 3: всего 10000, обнаружено 9929 (0.993), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 9940 (0.994), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 9983 (0.998), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 9984 (0.998), исправлено 0 (0.000)

Матрица 4x5, групп паритетов: 3, n=37
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 4185 (0.418)
  ошибок 3: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 9998 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)

Матрица 2x10, групп паритетов: 2, n=32
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 3: всего 10000, обнаружено 9950 (0.995), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 9959 (0.996), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 9993 (0.999), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 9990 (0.999), исправлено 0 (0.000)

Матрица 2x10, групп паритетов: 3, n=43
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 4517 (0.452)
  ошибок 3: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 9999 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)

Матрица 2x5x2, групп паритетов: 2, n=34
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 2506 (0.251)
  ошибок 3: всего 10000, обнаружено 9970 (0.997), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 9981 (0.998), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 9998 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 9996 (1.000), исправлено 0 (0.000)

Матрица 2x5x2, групп паритетов: 3, n=44
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 4374 (0.437)
  ошибок 3: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
```

```
Матрица 2x5x2, групп паритетов: 4, n=60
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 4056 (0.406)
  ошибок 3: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)

Матрица 2x5x2, групп паритетов: 5, n=76
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 3591 (0.359)
  ошибок 3: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)

Матрица 2x2x5, групп паритетов: 2, n=40
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 4110 (0.411)
  ошибок 3: всего 10000, обнаружено 9972 (0.997), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 9998 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 9999 (1.000), исправлено 0 (0.000)

Матрица 2x2x5, групп паритетов: 3, n=44
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 4452 (0.445)
  ошибок 3: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 9997 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)

Матрица 2x2x5, групп паритетов: 4, n=60
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 4147 (0.415)
  ошибок 3: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)

Матрица 2x2x5, групп паритетов: 5, n=76
  ошибок 1: всего 10000, обнаружено 10000 (1.000), исправлено 10000 (1.000)
  ошибок 2: всего 10000, обнаружено 10000 (1.000), исправлено 3542 (0.354)
  ошибок 3: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 4: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 5: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
  ошибок 6: всего 10000, обнаружено 10000 (1.000), исправлено 0 (0.000)
```

Вывод: приобрел практических навыков кодирования/де- кодирования двоичных данных при использовании кода Хемминга. приобретел практических навыков кодирования/дс- кодирования двоичных данных при использовании итеративных кодов .